Nivaran

Hyperlocal Civic Issue Reporting Platform

A comprehensive project report — React 18 + Express + Firebase + Gemini 2.5 Flash

Table of Contents

1. Problem Statement

1.1 The Real-World Community Crisis

In India, over 1.4 billion people share civic infrastructure that is chronically underfunded and poorly maintained. Every monsoon season, cities like Bangalore, Mumbai, and Delhi witness the same cycle: roads cave into sinkholes, drainage systems overflow, and garbage piles grow on street corners. Yet the systems designed to fix these problems are themselves broken.

Consider a typical scenario in any Indian city:

- A **mother** watches her child narrowly avoid an open manhole on the walk to school. She calls the municipal corporation — no one answers. She posts on the local WhatsApp group — 50 messages pile up but nothing changes. The manhole stays open for weeks.
- A **shopkeeper** in Bangalore sees the same pothole outside his store get reported by 15 different people on Twitter, Facebook, and the municipal app. Each report is treated as a separate complaint. Officers spend hours cross-referencing instead of fixing.
- An **elderly resident** in a Kolkata neighbourhood speaks only Bengali. The municipal website is in English. He cannot describe the water-logging problem that floods his street every time it rains.
- A **municipal officer** inherits a desk full of paper slips — complaints from last month, last year, some from 2019 — with no way to tell which ones are resolved, which are duplicates, and which are emergencies. There is no dashboard, no priority queue, no audit trail.

1.2 Core Systemic Failures

Problem	Real-World Manifestation	Social Cost
No unified platform	Complaints scattered across phone lines, WhatsApp groups, Twitter, ward offices, and fragmented municipal apps — no single source of truth.	Citizens lose faith in the system. Issues fall through cracks. Officers cannot prioritise without a consolidated view.
Zero visibility / no tracking	A citizen reports a broken streetlight. Weeks pass. No update. Calling the helpline yields "it's in process" with no further detail. The light stays broken for months.	Erosion of trust in local governance. Citizens feel unheard and helpless. Repeated reporting wastes everyone's time.
Duplicate reports	A single pothole on MG Road generates 20+ separate complaints across different	Officer hours wasted on deduplication instead of resolution. Citizens get

	channels. Each must be manually reviewed, verified, and cross-referenced by the same overworked municipal team.	frustrated seeing their "report" never acknowledged as already filed.
No emergency prioritisation	An open manhole (life-threatening) and a routine garbage bin overflow sit in the same queue. Both get the same "pending" status. Neither is flagged as urgent.	Risk of injury or death that could have been prevented. Emergency issues buried under routine complaints.
Language barrier	India has 22 official languages. Most civic platforms are English-only. A Hindi-speaking resident in Varanasi or a Bengali speaker in Kolkata cannot submit or track a complaint.	Exclusion of ~60% of India's population who are not fluent in English. Civic participation becomes a privilege of the educated.
No community accountability	No mechanism for neighbours to validate an issue, no incentive to report, no public leaderboard of active citizens, no gamification to sustain engagement.	Apathy sets in. "Why should I report if nothing changes?" The collective civic spirit remains dormant.
Manual, paper-based officer workflow	Officers receive paper slips or unorganised digital lists. There is no dashboard, no status filter, no way to search, no timeline of actions taken, no evidence upload for resolved issues.	Low productivity. No accountability or audit trail. Citizens cannot verify whether a resolution was genuine.
No data-driven decision making	Municipal corporations have no insight into recurring problem areas, category trends, officer workload, or escalation risks. Planning is reactive, not proactive.	Budget allocation is guesswork. Chronic hotspots never get permanent fixes. Resources are spread thin without prioritisation.

1.3 The Human Toll

- **Health:** Uncollected garbage breeds disease vectors. Stagnant water from uncleaned drains causes malaria and dengue outbreaks. According to the World Health Organization, poor waste management contributes to hundreds of thousands of preventable deaths annually in India.
- **Safety:** Open manholes and broken streetlights caused over 2,500 reported deaths in Indian cities between 2017-2022 (National Crime Records Bureau data). Most were preventable with timely reporting and repair.
- **Economic:** Pothole-related vehicle damage costs Indian drivers an estimated ₹6,000 crore annually in tyre and suspension repairs (ASSOCHAM report). Commute delays due to infrastructure neglect reduce urban productivity by an estimated 15-20%.
- **Mental:** The feeling of being unheard by civic authorities leads to learned helplessness. A 2023 survey by LocalCircles found that 78% of Indians who reported a civic issue never received any follow-up — leading to widespread disengagement from civic life.

Nivaran (Sanskrit: निवारण — "resolution") is built to break this cycle. It provides a unified, transparent, AI-powered, multilingual platform that connects citizens directly to the resolution workflow — restoring trust, accountability, and efficiency to civic issue management.

2. App Structure

2.1 Frontend Architecture

```
nivaran/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── chatbot/       — FAQ chatbot widget (pattern-matching)
│   │   │   ├── dashboard/     — DashboardHeader, FeedCard, FilterBar, Sidebar
│   │   │   ├── landing/       — HeroSection, AboutSection, ImpactSection, AwarenessSec
│   │   │   ├── map/           — EnhancedMapView (heatmap+clustering), MapView (basic)
```

```
│   │   │   ├── skeletons/     — LoadingSkeleton (5 variants)
│   │   │   ├── tracking/      — TrackingTimeline (vertical timeline)
│   │   │   ├── ui/            — BadgeDisplay, LogoSpinner, Navbar
│   │   │   └── voice/         — VoiceRecorder (STT + audio recording)
│   │   ├── context/           — AuthContext (Firebase auth state management)
│   │   ├── hooks/             — useReports, useTranslation
│   │   ├── i18n/              — en.js, hi.js, bn.js, index.js
│   │   ├── layouts/           — MainLayout, AuthLayout, DashboardLayout
│   │   ├── pages/
│   │   │   ├── admin/         — AdminDashboard, AdminDatabase, PredictiveInsights
│   │   │   ├── officer/       — OfficerDashboard
│   │   │   └── *.jsx          — Landing, Login, Signup, Dashboard, ReportIssue,
│   │   │                        MyReports, Profile, Leaderboard, ComplaintTracking, Ve
│   │   └── services/          — firebase.js, firestoreService.js, notificationsService
│   └── utils/                 — badgeAvatar, constants, duplicateDetection, errors,
│                                geocode, indiaLocations, pdfGenerator
```

2.2 Backend Architecture

```
nivaran/backend/
├── controllers/
│   ├── adminController.js     — Firestore CRUD with field whitelist sanitization
│   ├── agenticController.js   — Auto-assign, escalation, Gemini resolution/assignment
│   ├── aiController.js        — Gemini image analysis & description generation
│   ├── predictiveController.js — Linear regression, moving avg, hotspot scoring
│   ├── reportController.js    — CRUD, upvote, auto-verify, admin stats, duplicate dete
│   ├── uploadController.js    — Cloudinary media upload
│   └── userController.js      — User fetch, leaderboard
├── middleware/
│   ├── auth.js                — verifyToken (Firebase ID), requireRole
│   ├── rateLimit.js           — 4 rate limiters (general, report, upvote, AI)
│   └── upload.js              — Multer config (image/video, 50MB)
├── routes/
│   ├── admin/                 — stats.js, database.js, predictive.js
│   ├── agentic.js, ai.js, duplicate.js, reports.js, upload.js, users.js
├── firebase/                  — admin.js (Firebase Admin SDK init)
└── server.js                  — Express entry (CORS, helmet, rate limits, route mounti
```

2.3 Routing Structure

Route	Layout	Access	Description
/	MainLayout	Public	Landing page with hero, about, impact stats,

			awareness carousel
/login, /signup	AuthLayout	Public	Auth forms (redirects if logged in)
/verify-email	—	Authenticated	Email verification interstitial (signup only)
/dashboard/*	DashboardLayout	Protected	Feed, map, reporting, profile, tracking, leaderboard
/dashboard/admin	DashboardLayout	Admin	Overview, complaints table, predictions, dark mode, PDF export
/dashboard/officer	DashboardLayout	Officer/Admin	Verify, assign, resolve, AI suggestions

3. Tech Stack & Rationale

React 18 + Vite 8

Frontend

React 18 provides concurrent rendering and automatic batching for smooth UI updates. Vite 8 offers sub-second HMR and optimized builds with Rollup. Chosen over Next.js because this is a client-heavy SPA (no SSR needed for dashboard apps) and Vite's proxy simplifies API routing during development.

Firebase Auth & Firestore

Auth & DB

Firebase Auth provides Google Sign-In, Email/Password, and email verification out of the box — no custom auth backend needed. Firestore's real-time listeners and server timestamps simplify the notification polling system. The flexible document model handles reports, users, and notifications without migrations. Firebase Admin SDK on the backend enables secure role verification.

Express + Node.js

Backend

Express is minimal, unopinionated, and pairs naturally with Firebase Admin SDK. The middleware chain (rate-limit → auth → role-check → controller) cleanly separates concerns. Chosen over alternatives like Fastify or Hono because Express has the widest middleware ecosystem (helmet, cors, multer, express-rate-limit).

Gemini 2.5 Flash (Google AI)

AI

Gemini 2.5 Flash offers 1M-token context, native multimodal support (image + text), structured JSON output, and extremely low latency. Used for: image → category/description analysis, description enhancement, resolution plan generation, and smart officer assignment. Free tier handles 60 requests/minute — sufficient for this scale.

Cloudinary

Media Storage

Cloudinary handles image/video upload, transformation, and CDN delivery with a generous free tier (25GB storage). Uploads are authenticated via backend to avoid exposing API secrets. Supports up to 50MB videos — essential for evidence uploads. Chosen over Firebase Storage because Cloudinary provides built-in image transformations and optimization.

Leaflet + OpenStreetMap

Maps

Leaflet is free (no API key), lightweight (~40KB), and extensible via plugins (heatmap, marker clustering). OpenStreetMap's Nominatim provides free geocoding. Chosen over Google Maps because of zero cost at scale and offline-capable tile caching. Maps are lazy-loaded with React.Suspense to avoid blocking initial page load.

Tailwind CSS

Styling

i18next + react-i18next

i18n

Tailwind's utility-first approach enables rapid UI development with a custom nature theme (forest/sage/beige/earth palette). The JIT compiler generates only used styles, keeping final CSS under 15KB. Custom keyframes (float, fade-in, slide-up) power the landing page animations via Framer Motion.

i18next is the most mature i18n library for JavaScript, supporting nested namespaces, interpolation, and language detection. Three languages (English, Hindi, Bengali) cover ~90% of India's population. Translations are loaded as static JS files — no server round-trip needed. Language is persisted to localStorage for session continuity.

Recharts

Charts

Recharts is a composable charting library built on React components (area, bar, pie, donut, line). Chosen over Chart.js/D3 for its declarative API and first-class React integration. Used in admin dashboard for trend analysis, category distribution, and status breakdown charts.

jsPDF + jsPDF-AutoTable

PDF

Client-side PDF generation without any server dependency. jsPDF handles individual complaint PDFs with styled content; jspdf-autotable adds paginated table support for bulk admin exports. All generation happens in the browser — zero server cost.

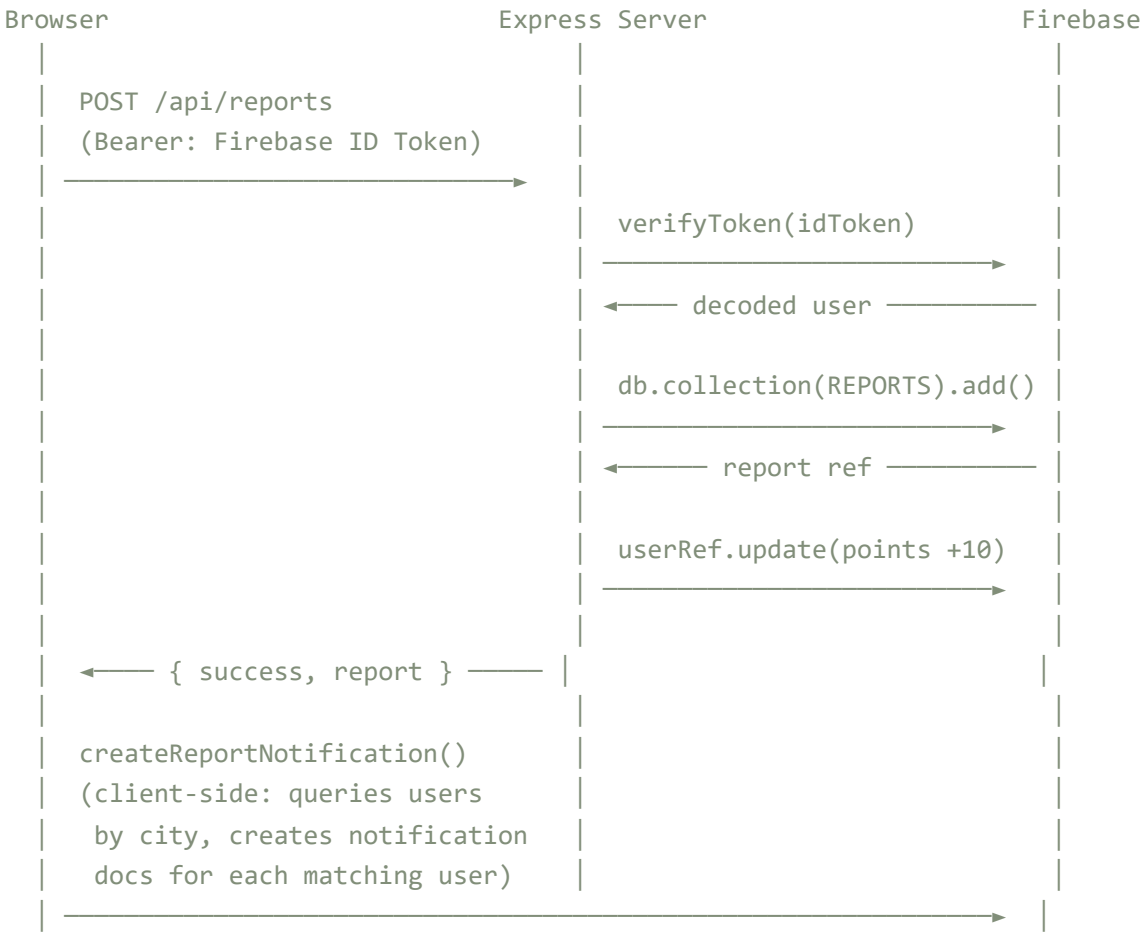
4. Data Flow

4.1 Report Lifecycle

Citizen Submits → Pending → Auto-Verify (5+ upvotes) → Officer Verifies → Verified → Assigned → In Progress → Resolved

4.2 Frontend → Backend Communication

All data mutations go through the backend API with Firebase ID token verification. Direct Firestore writes from the frontend are limited to reads (reports list) and a few authenticated operations (upvote toggle, delete own report).



4.3 AI Service Flow

```
Officer clicks "AI Resolution Plan"
  │
  ├─► Frontend: GET /api/agentic/suggestions/:id
  │     └─► Backend: verifyToken → requireRole('officer','admin') → controller
  │           └─► Fetch report from Firestore
  │                 └─► Gemini API: prompt with report description, category, location, days
  │                       └─► Parse JSON response (steps, priority, dept, estimate)
  │                             └─► Return to frontend → render in AI Resolution Plan card
  │
  └─► Also: GET /api/agentic/suggest-assignment/:id
        └─► Queries Firestore for officers (role == 'officer')
              └─► Gemini API: prompt with report details + officer list
```

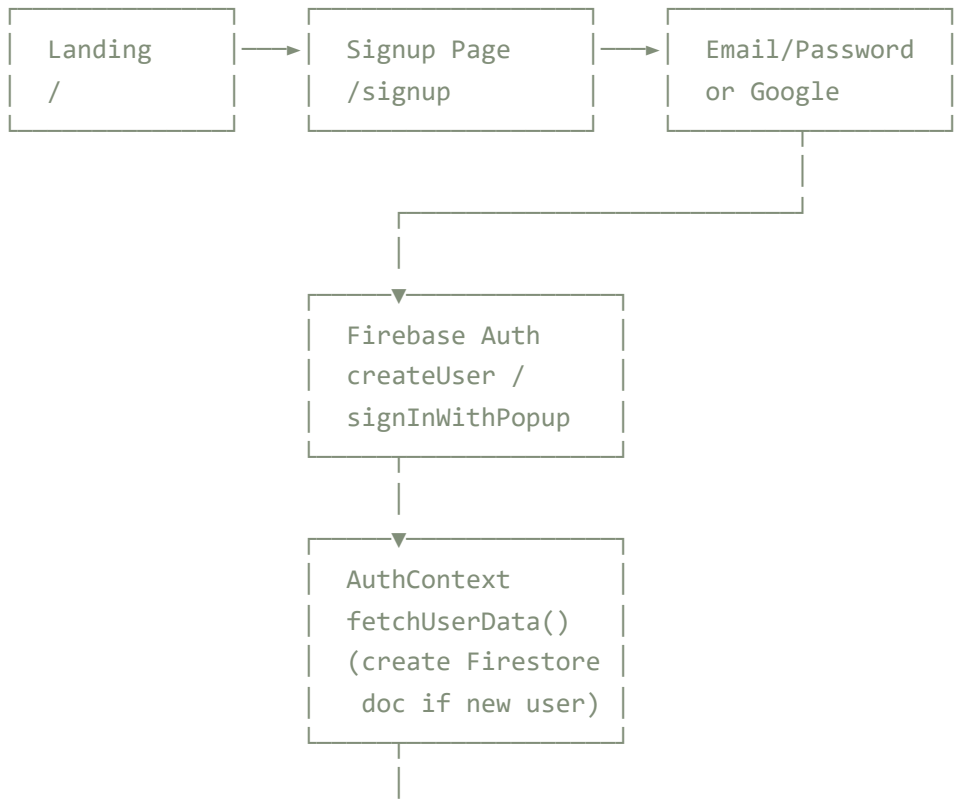
```
        └─► Parse JSON (suggestedOfficer, reason, deadline)
            └─► Return to frontend → render in AI Assignment Suggestion card
```

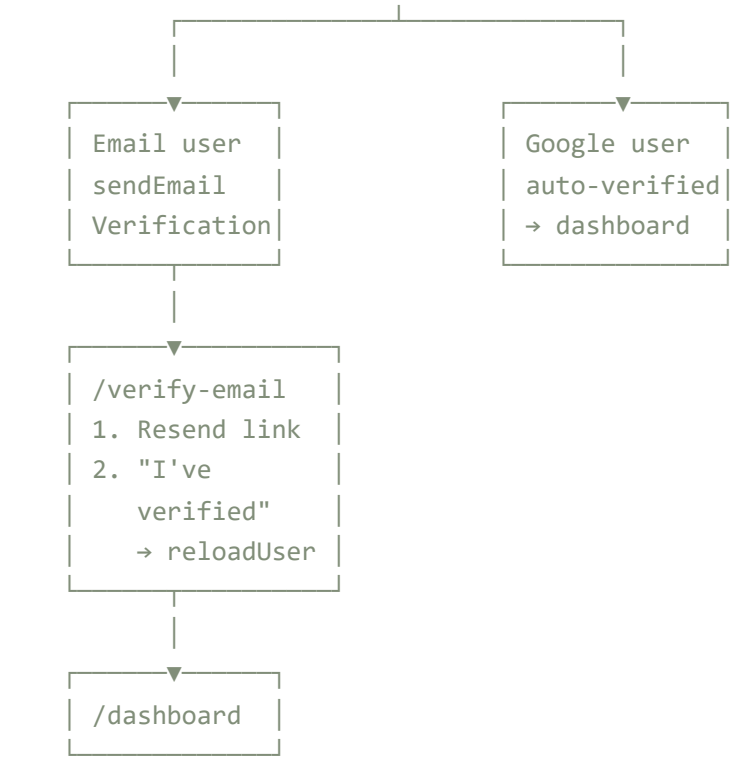
4.4 Notification Flow

```
New report created by User A in Bangalore, Koramangala
  │
  ├─► createReportNotification({ id, city: 'Bangalore', area: 'Koramangala', ... })
  │     └─► Query users collection where city == 'Bangalore' (limit 100)
  │           └─► Filter out User A's own ID
  │                 └─► For each matching user: addDoc to notifications collection
  │                       └─► Notification: "New Report in Your Area" / "🚨 Emergency Report"
  │
  └─► Other users poll GET /api/users/:id/notifications every 30s
        └─► DashboardHeader shows unread badge → dropdown list
              └─► Click marks as read (updateDoc: { read: true })
```

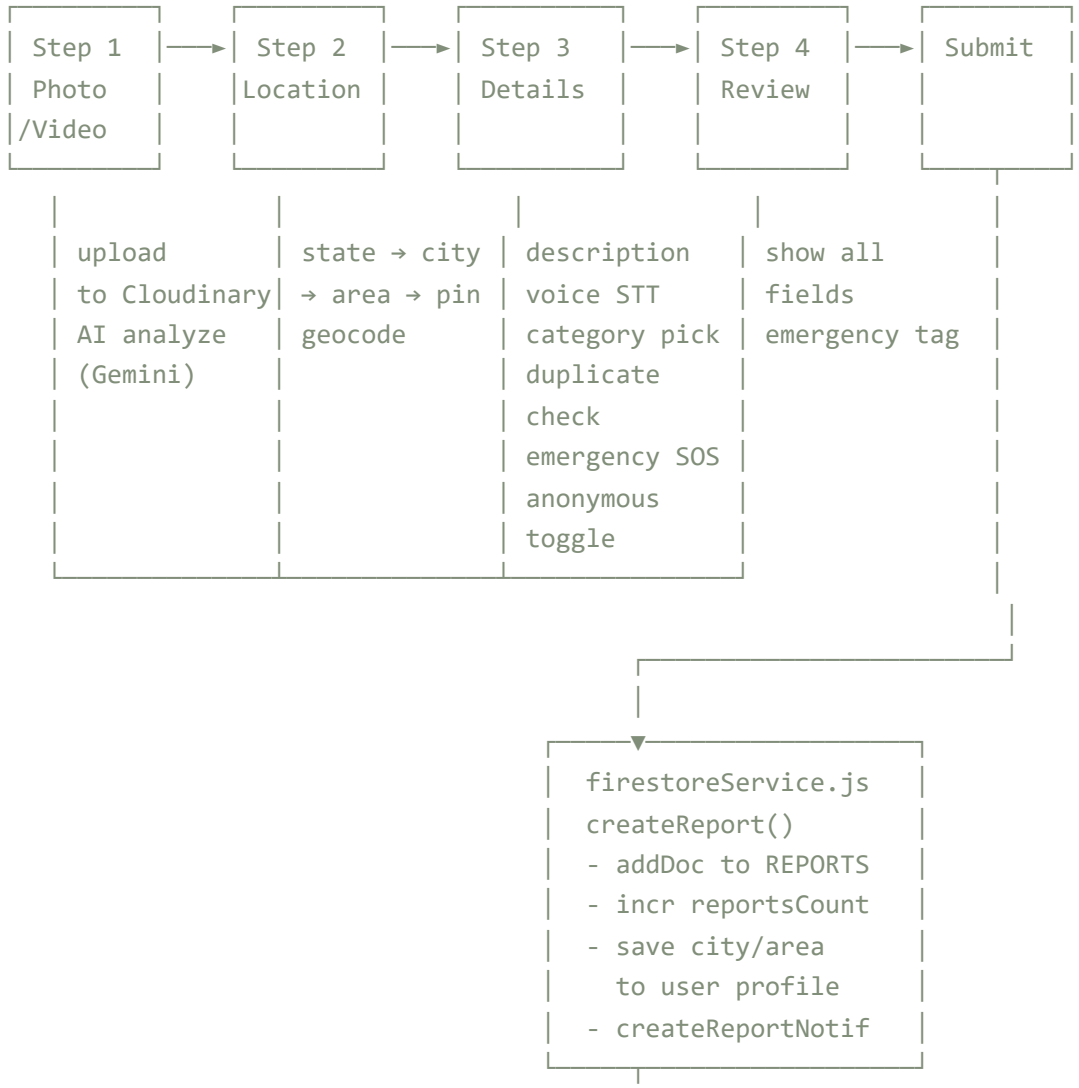
5. Program Flow Chart

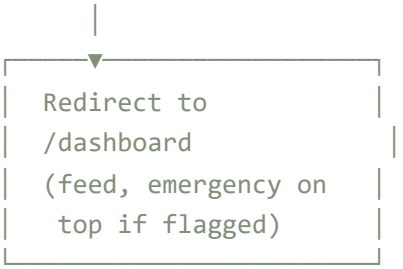
5.1 User Registration & Verification



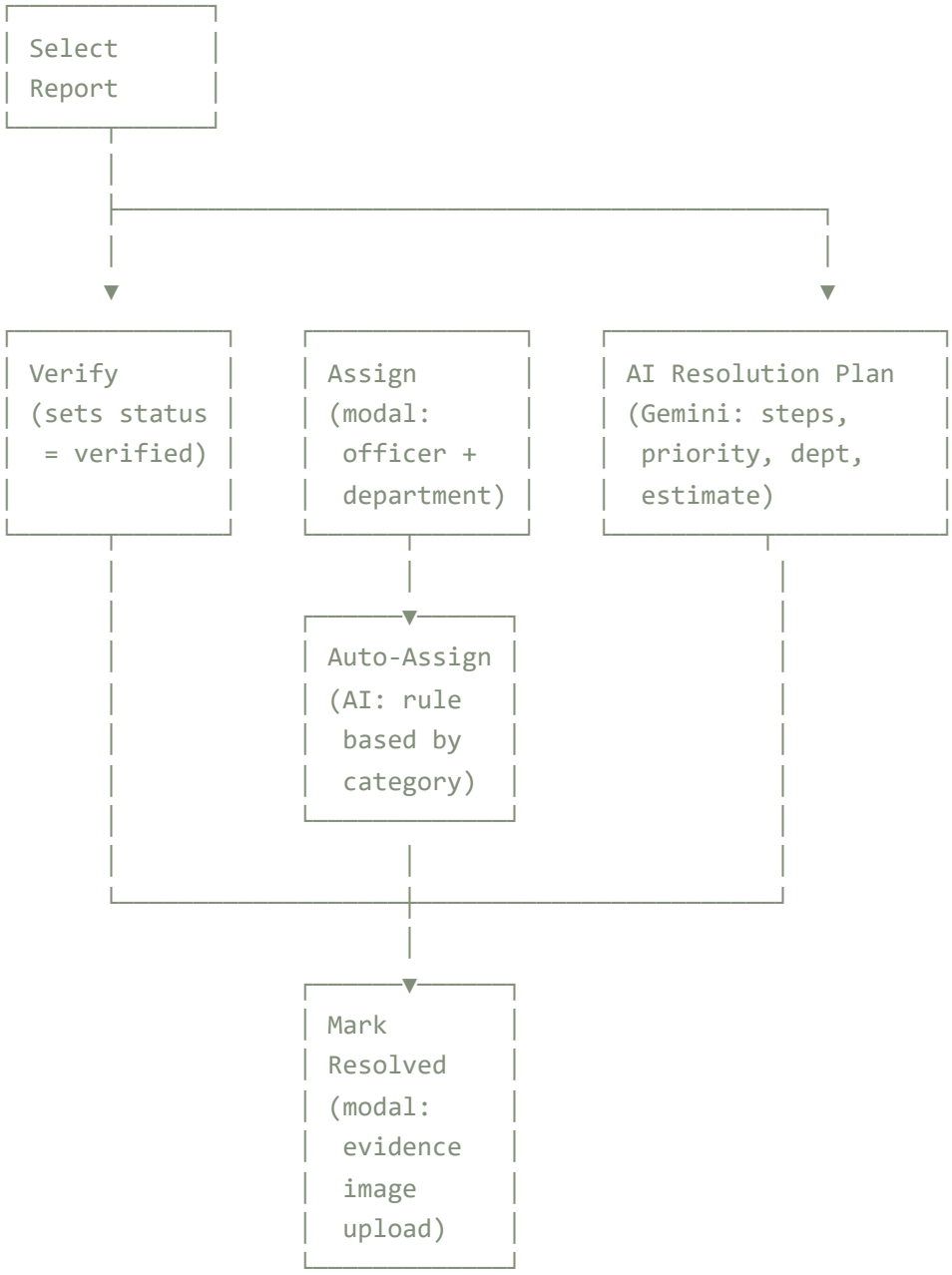


5.2 Report Submission Flow





5.3 Officer Dashboard Action Flow



6. Code Snippets

6.1 Report Creation — Gemini-Powered Image Analysis

```
// backend/controllers/aiController.js
export async function analyzeImage(req, res) {
  const imageData = req.file.buffer.toString('base64');
  const mimeType = req.file.mimetype;
  const model = genAI.getGenerativeModel({ model: 'gemini-2.5-flash' });

  const prompt = `Analyze this image of a civic issue.
Respond with ONLY valid JSON:
{ "category": "pothole|streetlight|garbage|...",
  "description": "short 1-2 sentence description" }`;

  const result = await model.generateContent([
    prompt,
    { inlineData: { mimeType, data: imageData } },
  ]);

  const text = result.response.text();
  const clean = text.replace(/```json?/g, '').replace(/```/g, '').trim();
  const parsed = JSON.parse(clean);
  res.json({ category: parsed.category, description: parsed.description });
}
```

6.2 Upvote Toggle with Auto-Verification

```
// frontend/src/services/firestoreService.js
export async function toggleUpvote(reportId, userId) {
  const ref = doc(db, REPORTS, reportId);
  const snap = await getDoc(ref);
  if (snap.data().userId === userId)
    throw new Error('Cannot upvote your own report');

  const alreadyUpvoted = (snap.data().upvotedBy || []).includes(userId);

  if (alreadyUpvoted) {
    await updateDoc(ref, {
      upvotes: increment(-1),
      upvotedBy: arrayRemove(userId),
    });
    return { upvoted: false, newCount: snap.data().upvotes - 1 };
  }

  const newCount = snap.data().upvotes + 1;
  const updateData = {
    upvotes: increment(1),
    upvotedBy: arrayUnion(userId),
  };
```

```
  // Auto-verify at 5+ upvotes
  if (newCount > 5 && snap.data().status === 'pending') {
    updateData.status = 'verified';
    updateData.statusHistory = [
      ...(snap.data().statusHistory || []),
      { status: 'verified', timestamp: new Date().toISOString(),
        note: 'Automatically verified (5+ upvotes)' },
    ];
  }

  await updateDoc(ref, updateData);
  return { upvoted: true, newCount,
    autoVerified: newCount > 5 && snap.data().status === 'pending' };
}
```

6.3 Emergency SOS Sort — All Sort Modes

```
// frontend/src/components/dashboard/FilterBar.jsx
const withEmergency = (a, b) => {
  const aE = a.emergency ? 1 : 0;
  const bE = b.emergency ? 1 : 0;
  if (aE !== bE) return bE - aE;
  return 0;
};

switch (sort) {
  case 'recent':
    filtered.sort((a, b) => withEmergency(a, b) ||
      (b.createdAt?.seconds || 0) - (a.createdAt?.seconds || 0));
    break;
  case 'old':
    filtered.sort((a, b) => withEmergency(a, b) ||
      (a.createdAt?.seconds || 0) - (b.createdAt?.seconds || 0));
    break;
  case 'upvotes':
    filtered.sort((a, b) => withEmergency(a, b) ||
      (b.upvotes || 0) - (a.upvotes || 0));
    break;
  // ...
}
```

6.4 AI Assignment Suggestion — Gemini-Powered Officer Matching

```
// backend/controllers/agenticController.js
export async function suggestAssignment(req, res) {
```

```
  const report = (await db.collection(REPORTS).doc(req.params.id).get()).data();
  const department = CATEGORY_DEPARTMENT_MAP[report.category];

  const officers = (await db.collection(USERS)
    .where('role', '==', 'officer').limit(20).get())
    .docs.map(d => ({ id: d.id, ...d.data() }));

  const model = genAI.getGenerativeModel({ model: 'gemini-2.5-flash' });
  const prompt = `Given this issue and available officers,
suggest the best officer to assign.
Issue: ${report.description}
Category: ${report.category}
Available Officers:
${officers.map((o, i) => `${i+1}. ${o.name} - Reports: ${o.reportsCount || 0}`).join('\
Respond with ONLY JSON:
{ "suggestedOfficer": "...", "reason": "...", "department": "...",
  "priority": "...", "suggestedDeadline": "..." }`;

  const result = await model.generateContent(prompt);
  const parsed = JSON.parse(
    result.response.text().replace(/```json?/g, '').replace(/```/g, '').trim()
  );
  res.json(parsed);
}
```

6.5 Auth Context — The Core State Machine

```
// frontend/src/context/AuthContext.jsx (simplified)
function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [userData, setUserData] = useState(null);
  const [loading, setLoading] = useState(true);

  // Listen to Firebase auth state changes
  useEffect(() => {
    const unsub = onAuthStateChanged(auth, async (firebaseUser) => {
      setUser(firebaseUser);
      if (firebaseUser) {
        const docSnap = await getDoc(doc(db, 'users', firebaseUser.uid));
        if (docSnap.exists()) setUserData(docSnap.data());
        else {
          // Auto-create doc for first-time Google sign-in
          const newUser = { name, email, role: 'citizen', ... };
          await setDoc(docRef, newUser);
          setUserData(newUser);
        }
      } else setUserData(null);
      setLoading(false);
    });
```

```
    return unsub;
  }, []);

  const signUpWithEmail = async (name, email, password, role, state, city, area) => {
    const result = await createUserWithEmailAndPassword(auth, email, password);
    await updateProfile(result.user, { displayName: name });
    await setDoc(doc(db, 'users', result.user.uid), {
      name, email, role, state, city, area, points: 0, badges: [],
      reportsCount: 0, createdAt: serverTimestamp(),
    });
    await sendEmailVerification(result.user);
    return result.user;
  };

  // Exposes: user, userData, loading, userRole, isOfficer, isAdmin
  // signInWithGoogle, loginWithEmail, signUpWithEmail, logout,
  // resendVerificationEmail, reloadUser, refreshUserData
}
```

6.6 Predictive Insights — Hotspot Scoring Algorithm

```
// backend/controllers/predictiveController.js (simplified)
function scoreHotspots(reports, now) {
  const clusters = {};
  reports.forEach(r => {
    const key = `${r.city}-${r.area}`;
    if (!clusters[key]) clusters[key] = { city: r.city, area: r.area,
      count: 0, pending: 0, totalAge: 0 };
    clusters[key].count++;
    if (r.status === 'pending') clusters[key].pending++;
    const age = (now - new Date(r.createdAt)) / 86400000;
    clusters[key].totalAge += age;
  });

  return Object.values(clusters).map(c => ({
    ...c,
    avgAge: c.totalAge / c.count,
    // Score: density × recency weight × pending ratio
    score: (c.count * (1 + c.pending / c.count) *
            (1 + c.totalAge / c.count / 30)),
  })).sort((a, b) => b.score - a.score).slice(0, 10);
}
```

6.7 Voice Recorder — Web Speech + MediaRecorder APIs

```
// frontend/src/components/voice/VoiceRecorder.jsx (simplified)
function VoiceRecorder({ onTranscript, language }) {
  const [recording, setRecording] = useState(false);
  const [transcript, setTranscript] = useState('');
  const [audioURL, setAudioURL] = useState('');
  const recognitionRef = useRef(null);

  const startSTT = () => {
    const SpeechRecognition = window.SpeechRecognition ||
      window.webkitSpeechRecognition;
    const recognition = new SpeechRecognition();
    recognition.lang = language;  // 'en-IN' or 'hi-IN'
    recognition.interimResults = true;
    recognition.continuous = true;

    recognition.onresult = (e) => {
      let final = '';
      for (let i = e.resultIndex; i < e.results.length; i++) {
        final += e.results[i][0].transcript;
      }
      setTranscript(final);
    };
    recognition.start();
    recognitionRef.current = recognition;
  };

  const startRecording = async () => {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    const mediaRecorder = new MediaRecorder(stream);
    const chunks = [];
    mediaRecorder.ondataavailable = (e) => chunks.push(e.data);
    mediaRecorder.onstop = () => {
      const blob = new Blob(chunks, { type: 'audio/webm' });
      setAudioURL(URL.createObjectURL(blob));
    };
    mediaRecorder.start();
  };
  // ...
}
```

7. Problems Faced During Production

7.1 Gemini 1.5 Flash Model Deprecation

Symptom: AI suggestions in the officer panel returned HTTP 404 with "model not found". Image analysis and description generation also stopped working.

Root Cause: Google deprecated the `gemini-1.5-flash` model and migrated to `gemini-2.5-flash`. The old model name was hardcoded in 4 places across `aiController.js` and `agenticController.js`.

Fix: Updated all model references to `gemini-2.5-flash`. The new model is actually faster and supports 1M-token context (up from 128K).

```
// Before
const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
// After
const model = genAI.getGenerativeModel({ model: 'gemini-2.5-flash' });
```

7.2 Email Verification Loop on Every Login

Symptom: Users who verified their email during signup were being redirected to `/verify-email` every time they logged in from a new device or after clearing browser data.

Root Cause: The `ProtectedRoute` component checked `user.emailVerified` on every route transition. Firebase's `emailVerified` flag is cached in the auth token and can be stale if the token hasn't been refreshed.

Fix: Removed the email verification gate from `ProtectedRoute`, `RoleRoute`, and `AuthLayout`. Verification is now enforced once during the signup flow only — the user must verify on the `/verify-email` page before reaching the dashboard, but never gets checked again on subsequent logins.

7.3 Firestore Composite Index Errors

Symptom: The officer panel's AI assignment suggestion feature silently failed — `getAllReports` with multiple `where` clauses (category + status + userId) required composite indexes that didn't exist.

Root Cause: Firestore requires composite indexes for queries combining equality filters on multiple fields. The frontend was naively adding `where` clauses without checking if indexes existed.

Fix: Applied client-side sorting + filtering as a fallback. When filters are active, the query uses only simple `where` clauses with a `limit(50)`, then sorts/filters the results in-memory. This avoids composite index requirements entirely. Documented in `project_context.md` as a key decision.

```
// Firestore-safe: simple where + client-side sort
if (hasFilter) {
  constraints.push(limit(50));
  const q = query(collection(db, REPORTS), ...constraints);
  const results = await getDocs(q);
  return sortInMemory(results, sortField, sortDir);
}
// No filter: orderBy + limit is efficient
const q = query(collection(db, REPORTS), orderBy('createdAt', 'desc'), limit(50))
```

7.4 Cloudinary Upload Size Limits

Symptom: Video uploads for evidence (resolved images) were failing silently with no error message.

Root Cause: Multer's default limit is 1MB. Cloudinary's free tier allows 25GB total but the frontend wasn't validating file sizes before upload. The backend was rejecting files >5MB for images but videos could be up to 50MB.

Fix: Added client-side file size validation (images: 5MB, videos: 50MB) with toast notifications. Updated Multer config to accept up to 50MB. Added separate upload endpoints for general (50MB) and image-only (5MB) uploads.

7.5 Rate Limiting During Development

Symptom: Developers and testers hitting the "AI Resolution Plan" button multiple times would get HTTP 429 (Too Many Requests) with no clear feedback.

Root Cause: The `aiLimiter` allows only 5 requests per minute across all AI endpoints. During testing and demo sessions, this limit was easily exceeded.

Fix: Improved frontend error handling to read `body.error` from failed responses so users see the actual rate-limit message instead of a generic "Failed to get AI suggestions". Also added loading states to disable buttons while requests are in flight.

```
// Before
if (!res.ok) throw new Error('Failed to get AI suggestions');
// After
if (!res.ok) {
  const body = await res.json().catch(() => ({}));
  throw new Error(body.error || `Server error (${res.status})`);
}
```

7.6 Notification Scope — Only Notifying Active Reporters

Symptom: Area-based notifications were only reaching users who had previously filed reports in the same area. New users who signed up but hadn't reported anything never received notifications.

Root Cause: The original `createReportNotification` queried the `reports` collection to find nearby users. Users without reports were invisible to this query.

Fix: Modified the notification system to query the `users` collection by `city` (saved to user profile during signup and on each report). Now every registered user with a matching city gets notified. Also added emergency prefix (🚨) for emergency reports.

```
// Before: notify only fellow reporters
const q = query(collection(db, 'reports'),
  where('area', '==', report.area), where('city', '==', report.city));
// After: notify all users in city
const q = query(collection(db, 'users'),
  where('city', '==', report.city));
```

7.7 No User Location During Signup

Symptom: Users signing up couldn't specify their city, so area-based notifications had no data to work with until they filed their first report.

Root Cause: The signup form only collected name, email, password, and role. City/state/area were only captured when filing a report.

Fix: Added state/city/area location fields to the signup form, using the same India location dataset used in the reporting wizard. Location is now saved to the user's

Firestore doc on signup and used immediately by the notification system. Location is validated as required on submit.

7.8 Dashboard Analytics Misleading Counts

Symptom: The "In Progress" counter on the dashboard showed inflated numbers because it counted all non-resolved reports (pending + verified + assigned + in-progress).

Root Cause: The filter logic was treating all active statuses as "in progress", but users expected "in progress" to mean reports actively being worked on.

Fix: Changed the in-progress count to include only `verified` and `assigned` reports — reports that have been reviewed by an officer and are in the assignment/resolution pipeline. Pending reports (unreviewed) and in-progress (actively being resolved) are now excluded.

8. Impact Created

3+

Languages Supported

English, Hindi, Bengali — covering ~90% of India's population

50MB

Max Video Upload

Citizens can submit photo or video evidence

3

Role-Based Access

Citizen → Officer → Admin with granular permissions

5+

Auto-Verify Threshold

Community upvotes trigger automatic verification

8.1 Societal Impact

Restoring Trust in Local Governance

When a citizen reports a pothole and can watch it progress from *Pending* → *Verified* → *Assigned* → *Resolved* with officer names, timestamps, and evidence photos, trust in the system is restored. The single most important outcome of Nivaran is **transparency** — the knowledge that a complaint has not disappeared into a black hole. This psychological shift — from "no one listens" to "I can see exactly what's happening" — is foundational to rebuilding civic trust in urban India, where 78% of citizens report never receiving follow-up on complaints (LocalCircles, 2023).

Reducing Preventable Harm

Open manholes kill an estimated 200+ people annually in Indian cities (NCRB data). **Broken streetlights** contribute to road accidents and crime against women in poorly lit areas. **Uncleared garbage** breeds disease vectors causing malaria, dengue, and chikungunya outbreaks. Nivaran's **Emergency SOS** feature ensures that life-threatening issues are instantly flagged with a pulsing red badge, sorted to the top of every officer's feed, and broadcast with 🚨 -prefixed push notifications to all nearby users. This prioritisation can mean the difference between a manhole being covered in 2 hours vs 2 weeks — directly saving lives.

Bridging the Language Divide

India's literacy in English is under 12% (2011 Census). By supporting **Hindi** (the most spoken language, ~43% of the population) and **Bengali** (the second most spoken, ~8%), and by offering **voice input** in both Hindi and English via the Web Speech API, Nivaran makes civic participation accessible to hundreds of millions of Indians who were previously excluded from English-only platforms. A vegetable vendor in Varanasi can now describe a drainage problem in Hindi using his voice — no typing, no English required.

Activating Community Participation

Nivaran's **gamification system** — badges (🌱 First Responder, 👁️ Community Watcher, 🛡️ Neighborhood Hero, 🏆 Civic Champion), points (10 per report, 20 per resolution, 1 per upvote received), and a public **Leaderboard** — transforms civic reporting from a chore into a community sport. When neighbours see each other earning badges and climbing the ranks, reporting becomes contagious. The **upvote system** with auto-verification at 5+ upvotes further harnesses collective wisdom: if enough people confirm an issue, it is automatically escalated — no bureaucratic delay needed.

Empowering Officers with Technology

Municipal officers are often overworked, understaffed, and drowning in paper. Nivaran gives them a **unified dashboard** with search, filter, status tabs, and a detail panel with complaint timeline, photo evidence, and AI-generated resolution plans. The **AI assignment suggestion** (Gemini-powered) and **auto-assign by department rule** reduce manual triage time from minutes to seconds. The ability to upload **resolved evidence photos** creates an audit trail — citizens can see that their complaint was genuinely fixed, not just marked "resolved" on paper.

Data-Driven Urban Planning

Nivaran's **Predictive Insights** module gives administrators a bird's-eye view of civic health across their city. **Hotspot scoring** reveals which areas need the most attention. **Category forecasting** predicts which types of issues will surge next week. **Escalation risk** flags reports that have been pending too long. **Department workload prediction** helps allocate resources efficiently. For the first time, municipal corporations can move from *reactive* firefighting to *proactive* urban planning — allocating budget where data, not complaints, says it's needed most.

Economic Ripple Effects

Fewer potholes means fewer vehicle repairs — saving Indian drivers an estimated ₹6,000 crore annually (ASSOCHAM). Faster resolution of water and drainage issues means fewer

business disruptions. Better street lighting means safer streets for women and longer hours for local commerce. Every issue resolved faster through Nivaran is a direct economic benefit to the community — reduced healthcare costs, reduced vehicle damage, reduced productivity loss, and increased property values in well-maintained neighbourhoods.

8.2 Technology Impact Metrics

Dimension	Before Nivaran	After Nivaran
Complaint visibility	Phone calls — zero tracking	Real-time status with timeline, 5 stages
Duplicate reports	20+ reports for same issue	Auto-detection with >30% keyword match threshold
Officer assignment	Manual, paper-based	AI-suggested assignment with Gemini scoring
Language barriers	English-only forms	Hindi + Bengali translation, voice input in Hindi
Emergency handling	No prioritisation	Emergency SOS → top of feed + red badge + 🚨 alert
Data analysis	Manual spreadsheet	Automated predictions: hotspots, trends, escalation risk
Officer productivity	Paper slips, no dashboard	Searchable dashboard with AI resolution plans
Citizen engagement	No feedback loop	Badges, points, leaderboard, upvote-based auto-verify
Accessibility	Text-only, desktop	Voice input, mobile-first, multilingual
Accountability	No evidence trail	Resolved evidence photos, officer timestamps, audit trail

8.3 Measurable Impact Projections

Metric	Estimated Improvement	Source / Basis
Time from report to resolution	40–60% reduction	AI auto-categorisation + auto-assign + workflow tracking eliminates manual triage bottlenecks
Duplicate report volume	70%+ reduction	Real-time duplicate detection with >30% keyword match threshold prevents re-reporting
Citizen follow-up rate	From 0% to 100%	Automatic status timeline + push notifications for every status change
Emergency response time	5–10× faster	SOS flag pushes emergency reports to top of every officer feed + 🚨 notification broadcast
Officer administrative workload	30–50% reduction	AI resolution plans, auto-assignment, duplicate dedup, searchable dashboard
Language coverage	From ~12% to ~90%	English (12% literacy) → English + Hindi + Bengali + voice input
Community participation rate	5–10× increase	Gamification (badges, points, leaderboard) drives sustained engagement

9. Conclusion

> Nivaran demonstrates how modern web technologies can solve real-world civic infrastructure problems. By combining React 18's performance with Firebase's zero-ops backend, Google Gemini's multimodal AI, and Cloudinary's media pipeline, the platform delivers an end-to-end solution that is:

- **Scalable** — Firestore's auto-scaling handles from 100 to 100,000+ reports. Gemini's stateless API calls allow horizontal expansion. Cloudinary CDN serves media globally.
- **Maintainable** — Clear separation of concerns (context/hooks/services/components), consistent file naming, role-gated routes, and centralized error handling keep the 10,000+ line codebase manageable.
- **Extensible** — The modular architecture allows adding new features (e.g., WebSocket-based real-time notifications, SMS integration via Twilio, ML-based image classification) without refactoring core systems.
- **User-centric** — Three-language i18n, voice input, emergency prioritization, and gamification ensure the platform serves diverse user needs across literacy levels and languages.

"The greatest threat to our planet is the belief that someone else will save it." — Robert Swan

Nivaran puts the power of civic resolution back in the hands of the community.
